

Sentinel: A Multi-Layer Web Security Architecture Using HTTP Request Handlers and HTTP Rules

By Anouar Moustarih, Quality Support Engineer, 4D Morocco

Technical Note 26-05

Table of Contents

Table of Contents	2
Abstract.....	4
Introduction	5
Prerequisites	6
Requirements	7
Architecture Overview	8
Project Shared Singleton Classes	8
Project Methods	9
Database Methods	9
Defense Techniques	10
Authentication and Brute-Force Protection	10
IP Allowlist and Blocklist	10
Rate Limiting	10
Header Validation	11
Web Application Firewall (WAF)	11
Honeypot URLs.....	11
Intrusion Detection and Prevention (IDS / IPS)	11
Panic Mode.....	12
Strict Response Headers.....	12
Defense Pipeline	13
The Ordered Sequence.....	13
Behavior of the Master Gate.....	13
Per-IP Sliding-Window Rate Limit	14
Global Rate Limit	14
Panic Mode.....	14
Sonar IDS and IPS. Scoring, Sniper, Drone.....	15
Purpose	15
Scoring Rubric	15
Queue Routing	15
Master Gate Interaction	16
Admin Endpoints	16
Authentication and Dynamic Passphrase Generation	17
Login Flow	17
Brute-Force Protection	17
Privilege Hierarchy	17
Dynamic Passphrase Generation	17
HTTPHandlers.json and HTTPRules.json	18
Web-Security Glossary	18
HTTP Request Handlers. Official Definition	19
HTTPHandlers.json Schema.....	19
When 4D Evaluates HTTPHandlers.json	19
Verbs and Privileges	19
How the Demo Application Uses HTTPHandlers.json	19
HTTP Rules.....	20
HTTPRules.json Schema.....	20
Every Rule in HTTPRules.json	21
	2

Response Headers Glossary	23
Configuration and Persistence	27
File Layout	27
Atomic Write Pattern	27
Hot-Reload Semantics	27
Dashboard and Frontend	28
Layout	28
Polling Cadence	28
Auth-Required Handling	28
Workers and Process Orchestration	30
Spawning	30
Shutdown	30
Health Reporting	30
Conclusion	31

Abstract

Sentinel is a 4D demo application that demonstrates how two 4D v21.0 LTS features—HTTP Request Handlers and HTTP Rules—can anchor a production-grade web security architecture. HTTP Request Handlers bind URL patterns to typed class functions, cleanly separating routing from business logic, while HTTP Rules inject context-appropriate security response headers without touching application code. Built on top of these primitives, Sentinel implements an eleven-gate defense pipeline that evaluates every inbound request in cost order: allowlist and blocklist lookups execute first; computationally heavier checks such as WAF pattern matching and rate limiting execute last. The pipeline covers CPU- and traffic-driven panic modes, header anomaly rejection, regular-expression WAF scanning for path traversal, SQL injection, and cross-site scripting payloads, honeypot deception for automated reconnaissance, and per-IP and global sliding-window rate limiting. Complementing the pipeline, the Sonar Intrusion Detection and Prevention System scores each request on additive behavioral signals—missing User-Agent headers, tool-based clients, spoofed browsers, and known attack patterns—and routes high-scoring requests to an immediate-block sniper queue while archiving lower-scoring requests for pattern correlation. Authentication is handled by a dedicated SentinelAuth class that validates a bcrypt-hashed static passphrase or a one-shot Dynamic Passphrase, and enforces per-IP brute-force lockout. All runtime state persists through atomic write patterns to JSON files under the project Data folder, and configuration reloads hot without a server restart. A vanilla-JavaScript dashboard provides live telemetry, defense toggles, attack simulators, and worker health monitoring.

Introduction

Modern web servers face a broad spectrum of automated threats—credential brute force, volumetric denial-of-service floods, injection attacks, and systematic reconnaissance by scanners probing for misconfigured paths. Addressing these threats individually with ad-hoc code leads to scattered, hard-to-audit logic. Sentinel takes a different approach: it uses the two declarative security primitives introduced in 4D v21.0 LTS—HTTP Request Handlers and HTTP Rules—as the structural foundation, then layers defense modules on top in a single, ordered pipeline.

This Technical Note walks through every layer of that architecture. It begins with the fifteen shared singleton classes that own discrete responsibilities—from IP list management and rate-limit token buckets to worker lifecycle and alert escalation—and the background workers that drive them. It then explains each of the eight defense techniques deployed in the pipeline, covering the attack each technique addresses and exactly how the demo application applies it. The Defense Pipeline section documents the eleven-gate execution order and the behavior of each gate under normal and panic conditions. The Sonar section details the additive scoring rubric, the sniper and drone queue routing cut-offs, and the three admin endpoints that expose engine state. The HTTPHandlers.json and HTTPRules.json section provides the full schemas, every registered route, and a per-rule explanation of the response headers and the threats each header mitigates. The document closes with configuration persistence, hot-reload semantics, dashboard polling cadences, and worker orchestration shutdown sequences.

Prerequisites

Readers should bring the following baseline familiarity before working through the document.

- Shared singleton classes.
- HTTP fundamentals. Verbs, headers, status codes.
- JSON syntax and basic regular expressions.
- Common web-attack categories. Denial of service, SQL injection, path traversal, header overflow.

Requirements

The following items must be in place before the demo application can be exercised end to end.

- **4D Version.** 21.0 LTS or higher.
- **Network.** Port 8044 must be open and not blocked by a local firewall or by network security policies.
- **Demo Application.** The localhost address (127.0.0.1) and the local network address (LAN or WAN) of the host computer must both be present in the allowlist.

Example:

```
"allowlist": {  
  "127.0.0.1": "Manual",  
  "192.168.XXX.YY": "Manual or via Dashboard"  
}
```

Important. Clicking the *Clear All* button in the *IP Blacklist* panel empties the *ip_lists.json* file. If this occurs, 127.0.0.1 and any other required addresses must be re-added, either manually or through the *allowlist* button on the dashboard.

Bug ACI0106333 (Windows Specific) On Windows, *Process activity.processes[0].cpuUsage* returns an inflated value compared to macOS. This telemetry discrepancy causes the CPU panic gate to trigger, rejecting inbound with an HTTP 503 status. A fix is in progress. The version containing the fix will be supplied as soon as it's available. macOS is unaffected.

Project Shared Singleton Classes

DoSGuard: Owns the eleven-gate defense pipeline. Exposes `authenticateRequest` as the entry point for the On Web Authentication database method and for the `_gate` class function called from public-facing handlers.

Sentinel: Implements the admin HTTP API surface of forty-plus endpoints. Includes status, toggles, attacks, panic, honeypot, workers, sonar, and dashboard CRUD class functions.

SentinelAuth: Owns login, logout, the auth-status check, brute-force lockout, and dynamic passphrase generation. Validates the bcrypt-hashed static passphrase.

Sonar. The intrusion detection and prevention engine. Scores each inbound request from 0 to 100. Routes high-score requests to the sniper queue and low-score requests to the drone queue.

Honeypot: Detects requests to decoy URL paths. Records the hit, bans the source IP for twenty-four hours, and raises a CRITICAL alert.

IPManager: Owns the blocklist, the allowlist, and the strike counters. Persists to `Data/ip_lists.json` through an atomic write.

ConfigManager: Owns the runtime configuration store. Reads and writes `Data/dosguard_config.json` with an atomic write pattern.

RequestLogger: Owns the in-memory request log. Two-thousand-entry ring buffer with periodic flush to a daily JSONL file on disk.

AlertManager: Owns the threshold-based alerts. Raises INFO, WARNING, and CRITICAL events with a sixty-second cooldown per alert key.

ProcessOrchestrator: Owns the worker lifecycle. Spawns workers at startup, broadcasts the shutdown signal at exit, and tracks heartbeats for the Workers panel.

RulesManager: Provides CRUD access to `HTTPRules.json` from the dashboard. Maintains the on-disk version for hot-reload.

HandlersManager: Provides CRUD access to `HTTPHandlers.json` from the dashboard. Maintains the on-disk version for hot-reload.

HeaderValidator: Validates inbound request headers. Checks header size, null bytes, and CRLF injection attempts.

FileUploadHandler: Handles file upload requests. Sanitizes inputs and forwards to the `_gate` class function for defense checks.

URLNormalizer: Canonicalizes inbound URLs to a normal form before the WAF inspects them.

URLCodec: Performs percent-decoding with multi-pass support to defeat nested encoding attempts.

Project Methods

CPUMonitor_Worker: Samples process CPU activity at one-second cadence. Flips the CPU panic flag when CPU stays above the configured threshold.

PanicWatchdog_Worker: Monitors the global request rate at five-second cadence. Auto-lifts traffic panic mode when the rate drops below the lift threshold.

LogFlusher_Worker. Flushes the RequestLogger ring buffer to the daily log file on disk at roughly five-second cadence.

HoneypotSweeper_Worker: Clears expired honeypot bans at sixty-second cadence. Honeypot bans last twenty-four hours.

Sonar_Sniper_Worker: Drains the sniper queue at one-second cadence. Issues a one-hour block on the source IP and appends one line per kill to Data/sonar_kills.jsonl.

Sonar_Drone_Worker. Drains the drone queue at ten-second cadence. Archives the request to Data/sonar_archive.jsonl without blocking the source IP.

Attack_CPUArmageddon: Internal attack simulator. Floods /heavy from a worker thread to push CPU above the panic threshold.

Attack_Bandwidth: Internal attack simulator. Multi-vector traffic with recon and traversal probes to exercise rate limits and WAF.

Attack_HeaderOverflow: Internal attack simulator. Oversized, malformed, and CRLF-injected headers to exercise the HeaderValidator.

Attack_GhostFlood: Internal attack simulator. Fifty-IP distributed flood to exercise the global rate limit and the blocklist auto-promotion logic.

Startup: Initializes the shared singleton classes, loads the on-disk config, registers the workers with ProcessOrchestrator, and starts the web server.

Database Methods

On Startup: Fires once when the 4D project opens. Spawns the Startup project method in a new process

On Web Authentication: Fires for each inbound HTTP request before the route is matched. Runs Sonar.intercept, then DoSGuard.authenticateRequest, then sets the session privilege.

On Exit: Fires when the 4D project closes. Stops the web server first, then signals shutdown to every worker, then issues KILL WORKER for each registered worker.

Defense Techniques

This section explains every defense technique used in the demo application as a general web-security concept. The implementation specifics live in the sections that follow. Each technique below names the defense, the attack it addresses, and how the demo application applies it.

Authentication and Brute-Force Protection

Authentication confirms the identity of the client before granting privileged access. Brute-force protection caps the number of failed authentications attempts a single source may make in a time window. The two controls are paired because authentication on its own does not stop password guessing.

Attack addressed: Credential brute force, where an attacker tries many passwords against a known account. Also, credential stuffing, where the attacker uses passwords leaked from other services.

How the demo application applies it. The SentinelAuth class validates a bcrypt-hashed static passphrase or a one-shot dynamic passphrase. After five failed login attempts within fifteen minutes, the source IP is locked out for thirty minutes. The lockout state lives in memory only.

IP Allowlist and Blocklist

An allowlist is a list of source IP addresses that bypass defense checks. A blocklist is a list of source IP addresses that are denied all access. Allowlists are useful for administrator workstations and trusted monitoring tools. Blocklists hold attackers and repeat abusers.

Attack addressed: Once an attacker is identified by any defense layer, the blocklist prevents the attacker from continuing. Strike counters cause repeated offenders to be permanently blocked.

How the demo application applies it. The IPManager class owns both lists. The lists persist to Data/ip_lists.json through an atomic write. The dashboard allows manual edits. A strike counter promotes a temporary block to permanent after five strikes.

Rate Limiting

Rate limiting restricts how many requests a single source may send within a time window. Two common variants exist. Per-IP rate limiting tracks each source address separately. Global rate limiting tracks aggregate traffic across all sources.

Attack addressed: Volume attacks that aim to exhaust server resources by sending more requests than the server can process. Includes denial of service floods, brute-force login attempts, and credential-stuffing campaigns.

How the demo application applies it. A per-IP token bucket caps each source at 100 requests per minute and 20 requests per five-second burst. A global aggregate counter caps total traffic at 10,000 requests per minute. Breach triggers a temporary block and increments the strike counter.

Header Validation

HTTP header validation rejects requests whose headers exceed a reasonable size, contain null bytes, or contain CRLF (carriage return + line feed) sequences. Such headers are usually attempts to smuggle a second request inside the first.

Attack addressed: Header overflow attacks that try to crash the server through oversized headers. Null-byte attacks that try to truncate string processing. CRLF injection that tries to inject extra headers or split the request into two.

How the demo application applies it. The HeaderValidator class checks header size against the configured maximum, default 8,192 bytes. It also rejects any header containing a null byte or a stray CRLF sequence. Violators receive HTTP 400 and the source IP is blocked for 300 seconds.

Web Application Firewall (WAF)

A Web Application Firewall inspects HTTP requests for known attack patterns before forwarding them to the application code. The WAF maintains a list of malicious patterns. Any request that matches a pattern is rejected with an HTTP 403 response.

Attack addressed: Injection attacks where the attacker embeds malicious payloads in URL parameters or paths. Includes path traversal, SQL injection, cross-site scripting, command injection, and eval-based code execution.

How the demo application applies it. The WAF inside the DoSGuard class scans the request URL for path traversal sequences, eval calls, script tags, union-select SQL fragments, and SQL injection tails. A match returns 403 with the matching category. Each rejection adds a strike to the source IP.

Honeypot URLs

A honeypot URL is a decoy path the server does not actually serve. Legitimate clients never request these paths. Any client that does is treated as malicious or as an automated scanner. The server records the hit and bans the source.

Attack addressed: Reconnaissance attacks where the attacker (often a bot) probes for known sensitive paths such as /.env, /.git/config, /wp-admin, or /admin.php. Discovery of such paths indicates misconfiguration on poorly secured servers.

How the demo application applies it. About twelve honeypot URLs are registered as routes in HTTPHandlers.json. Each maps to Honeypot.handleProbe. A hit logs a CRITICAL alert and bans the source IP for twenty-four hours.

Intrusion Detection and Prevention (IDS / IPS)

An Intrusion Detection System (IDS) inspects traffic for signs of malicious activity and emits alerts. An Intrusion Prevention System (IPS) goes further and blocks the offending request or source. Modern systems combine both roles in one engine.

Attack addressed: Targeted attacks where the attacker does not match any single pattern but presents multiple weak signals. Includes scripted scanning campaigns, credential stuffing, and reconnaissance by tools such as nmap or sqlmap.

How the demo application applies it. The Sonar class assigns each request a score from 0 to 100 based on additive signals. Scores at or above 80 trigger an immediate one-hour block, which is the IPS behavior. Scores between 1 and 79 are archived for correlation, which is the IDS behavior.

Panic Mode

Panic mode is an emergency state where the server stops processing normal traffic and accepts only requests from a small allowlist. The mode is used as a last resort when CPU, memory, or traffic volume cross a threshold the operator considers unsafe.

Attack addressed: Distributed flood attacks that overwhelm the per-IP rate limiter through sheer source diversity. Also covers situations where a misbehaving client or an internal bug causes resource saturation.

How the demo application applies it. The CPUMonitor_Worker samples process CPU at one-second cadence. Panic engages when CPU stays at or above 85% for 30 seconds and disengages when CPU drops below 50%. The operator can also trigger or lift panic manually from the dashboard.

Strict Response Headers

Strict response headers tell the browser how to render the response safely. Common headers include Content-Security-Policy (which scripts may run), X-Content-Type-Options (do not guess MIME types), X-Frame-Options (do not allow framing), and Referrer-Policy (limit referrer leakage).

Attack addressed: Cross-site scripting (XSS), clickjacking, MIME sniffing attacks, and information leakage through the Referer header.

How the demo application applies it. Every response gets baseline headers from the HTTPRules.json catch-all rule. Specific routes layer stricter headers on top. The full per-route policy is documented in the HTTPHandlers.json and HTTPRules.json section of this technote.

Defense Pipeline

The Ordered Sequence

DoSGuard.authenticateRequest runs eleven ordered checks. The first check that produces a verdict ends the pipeline.

The order is intentional. The cheapest and most decisive checks run first. Expensive checks (WAF, rate limit) run last.

Gate	Check	Action on match
1	Master gate (defenses.master)	If OFF, accept immediately. No further checks.
2	CPU panic flag	Reject non-allowlisted IPs with 503.
3	Traffic panic flag	Reject non-allowlisted IPs with 503. Auto-lift on expiry.
4	Header size limit (default 8192 bytes)	Reject with 400. Block source IP for 300 seconds.
5	Handler defense toggle	If OFF, accept and forward request to the matched handler.
6	IP allowlist	Accept. Skip remaining gates.
7	IP blocklist	Reject with 403.
8	Honeypot URL match	Record hit. Let handler render the 403 deception body.
9	WAF (URLNormalizer + URLCodec)	Reject with 403 on traversal, eval, script, SQLi.
10	Per-IP sliding-window rate limit	Reject with 429. Strike. Block on burst.
11	Global rate limit	Reject with 429. Raise alert.

Behavior of the Master Gate

The master gate is the first check. When defenses.master is false, the pipeline accepts immediately and skips every later check, including panic.

The CPU monitor and panic watchdog still update their flags during a master-off run. The dashboard panic indicator can therefore show ACTIVE while no enforcement occurs. The indicator is cosmetic during master-off mode.

Per-IP Sliding-Window Rate Limit

Each source IP gets a token bucket with two windows. The wide window covers sixty seconds. The burst window covers five seconds.

Defaults. 100 requests per minute. 20 requests per five-second burst. 300-second block on breach. Permanent block after five strikes.

Defaults live in Data/dosguard_config.json and are editable from the dashboard.

Global Rate Limit

A single aggregate counter tracks total accepted requests across all IPs over a rolling minute. The default ceiling is 10,000 requests.

On breach, the request returns 429 and an alert is raised. A pre-threshold WARNING fires at 50% of the ceiling so the operator sees the climb before the cap is reached.

Panic Mode

Panic mode rejects every request from a non-allowlisted source with HTTP 503. The mode is triggered automatically by the CPU monitor or manually from the dashboard.

Automatic CPU panic engages when CPU stays at or above 85% for 30 seconds. The mode lifts when CPU drops below 50% for the minimum duration.

Allowlisted IPs (for example the administrator workstation) bypass panic and retain access to the dashboard.

Sonar IDS and IPS. Scoring, Sniper, Drone

Purpose

Sonar inspects every request after the On Web Authentication database method runs. Inspection produces an integer score from 0 to 100.

The score routes the request to one of two queues. High scores receive immediate enforcement. Low scores receive archival for pattern correlation.

Scoring Rubric

Scoring is additive and capped at 100. Each signal adds points. The final score determines routing.

Signal	Points
Missing User-Agent header	+30
Already-blocked IP re-knocking	+80
URL contains path-traversal ../	+50
URL contains eval(	+50
URL contains <script	+50
URL contains union select	+50
URL contains '-- or "';-- (SQLi tail)	+50
Tool UA. curl, postman, wget, python-requests, go-http-client, powershell	+25 each
Spoofed browser. Mozilla UA without Sec-Fetch-* or Sec-Ch-Ua headers	+60
Bot UA. googlebot or bingbot	+50
Generic bot UA. crawler or spider	+40

Queue Routing

Routing is decided by score range. The cut-off is 80.

- Score 80 or higher. Request enters the sniper queue (cap 500). The Sonar_Sniper_Worker drains the queue, issues a one-hour block, and writes one line to Data/sonar_kills.jsonl.

- Score 1 to 79. Request enters the drone queue (cap 2000). The Sonar_Drone_Worker archives the request to Data/sonar_archive.jsonl without blocking.
- Score 0. Request is not queued. The intercepted counter still ticks.

Master Gate Interaction

When the master gate is OFF, Sonar short-circuits. The intercepted counter still increments, but no scoring or queuing occurs.

Admin Endpoints

Three administrator endpoints expose Sonar state. Each requires the sentinelAdmin privilege.

- /admin/sonar/stats. Current counters and last-seen score.
- /admin/sonar/queue. Up to 50 in-flight queued items.
- /admin/sonar/archive. Last 200 entries from the drone archive.

Authentication and Dynamic Passphrase Generation

Login Flow

Dashboard authentication is handled by `SentinelAuth.handleLogin`. The class function checks three credential sources in order.

- **Pending Dynamic Passphrase Generation (DPG) pool.** A per-IP code valid for five minutes after generation.
- **Active DPG.** A temporary passphrase visible to the administrator.
- **Static passphrase.** A bcrypt hash stored in `dosguard_config.json` under `auth.dashboardPassphrase`.

Brute-Force Protection

`SentinelAuth` tracks failed logins per IP across a fifteen-minute window. After five failures, the IP is locked out for thirty minutes.

Privilege Hierarchy

The demo application defines three privileges in `Project/Sources/roles.json`. Privileges nest from least to most powerful.

Privilege	Granted to	Scope
guest	Every incoming request	Public health, honeypot probes, file upload.
sonar	Set by the On Web Authentication database method on every request	DoSGuard and Honeypot handler class functions.
sentinelAdmin	Granted only by <code>SentinelAuth.handleLogin</code>	Full dashboard CRUD, panic, attack simulation.

Dynamic Passphrase Generation

DPG provides a one-shot login credential for emergency dashboard access. The mechanism avoids storing or sharing the long-lived static passphrase during incident response.

DPG generation is exposed at `/admin/auth/generate-passphrase` and is rate-limited to ten generations per minute per IP.

Example of a 4D class function call that initiates DPG.

```
$response: =cs.SentinelAuth.me.handleGeneratePassphrase($request)
```

Web-Security Glossary

The following terms appear throughout this section.

Term	Definition
User-Agent (UA)	The HTTP request header that identifies the client software. Browsers, curl, Postman, search-engine crawlers, and scripted clients each present a distinct UA string. The Sonar class classifies clients by parsing this header.
Content-Security-Policy (CSP)	A response header that tells the browser which scripts, styles, fonts, images, and frames the page is allowed to load. A strict CSP defeats most cross-site scripting attacks.
Cross-Origin Resource Sharing (CORS)	The mechanism that lets a browser fetch data from a server hosted at a different origin. Controlled by Access-Control-Allow-* response headers and the OPTIONS preflight request.
Honeypot URL	A decoy path the demo application does not actually serve. Any client that requests one is treated as malicious and banned for twenty-four hours.
Path traversal	An attack technique that uses ../ sequences inside a URL to read files outside the intended directory. The WAF rejects any URL containing the sequence.
SQL injection (SQLi)	An attack that injects SQL fragments into a query through a URL parameter. The WAF flags tokens such as union select and tail markers such as '-- or ';--.
MIME sniffing	A browser behavior that guesses the content type of a response instead of trusting the server-declared Content-Type. The header X-Content-Type-Options: nosniff disables this.
Clickjacking	An attack where a malicious page hides the target site inside a frame and tricks the end user into clicking on the framed page. The header X-Frame-Options: DENY blocks this.
Referrer-Policy	A response header that controls what the browser sends in the Referer header when following a link out of the page. Limits information leakage to third-party sites.
Permissions-Policy	A response header that disables browser features (camera, microphone, geolocation, etc.) the page does not need. Reduces the attack surface in case of cross-site scripting.

HTTP Request Handlers. Official Definition

From the 4D Doc Center. HTTP Request Handlers let the developer process specific HTTP requests through code in the application. Each handler binds a URL pattern to a class function. The configuration lives in Project/Sources/HTTPHandlers.json.

HTTPHandlers.json Schema

HTTPHandlers.json is a JSON array. Each entry describes one routable URL pattern.

The required keys are `regexPattern`, `verbs`, `class`, and `method`. The optional key is `comment`.

A minimal valid entry looks like the example below.

```
{
  "regexPattern": "^/admin/status$",
  "verbs": "GET",
  "class": "Sentinel",
  "method": "handleStatus"
}
```

4D evaluates the array in declaration order. The first matching `regexPattern` wins. Unmatched URLs fall through to the On Web Connection database method.

When 4D Evaluates HTTPHandlers.json

The 4D web server loads HTTPHandlers.json at startup and rereads it whenever the file is replaced on disk. No restart is required.

Evaluation order at runtime. TLS termination, then On Web Authentication, then handler match, then class function invocation, then HTTPRules.json header injection, then response.

Verbs and Privileges

The verbs string accepts any combination of GET, POST, PUT, DELETE, OPTIONS, HEAD, PATCH. Requests with non-listed verbs return 405.

Privileges are not declared in HTTPHandlers.json. Privileges are declared in roles.json against the class or class function name. The two files cooperate.

How the Demo Application Uses HTTPHandlers.json

The demo application registers about forty routes. The routes are grouped into categories by URL prefix.

Category	Pattern prefix	Class	Routes
Authentication	/admin/auth/*	SentinelAuth	5
Dashboard telemetry	/admin/{status,logs,workers,cpu}	Sentinel	4
Defense toggles	/admin/toggle/*	Sentinel	5
Attack simulation	/admin/attack/*	Sentinel	4
Panic mode	/admin/panic/*	Sentinel	3
Sonar admin	/admin/sonar/*	Sentinel	3
Honeypot admin	/admin/honeypot/*	Sentinel	2
Dashboard API	/api/dashboard/*	Sentinel	14
Public API	/api/public/*, /health, /heavy, /public	DoSGuard	4
File upload	/upload/*	FileUploadHandler	1
Honeypot probes	/.git/config, /.env, /admin.php, ...	Honeypot	1

The demo application does not catch routes through the On Web Connection database method. This makes the routing surface explicit.

HTTP Rules

HTTP Rules let the developer define security policies for HTTP responses through the addition of HTTP response headers. The configuration lives in `Project/Sources/HTTPRules.json`.

HTTPRules.json Schema

HTTPRules.json is a JSON array. Each entry has a regex selector and a header bag.

Required keys are `regexPattern` and `responseHeaders`. Either of `requestHeaders` or `responseHeaders` may be present.

A baseline rule that adds security headers to every URL is shown below.

```
{
  "regexPattern": "^/.*$",
  "responseHeaders": {
    "X-Content-Type-Options": "nosniff",
    "X-Frame-Options": "DENY",
    "Referrer-Policy": "strict-origin"
  }
}
```

Every Rule in HTTPRules.json

The table below explains every rule in the live HTTPRules.json file. One row per regexPattern entry.

regexPattern	Applies to	Headers added	Why
^/.*\$	Every URL (catch-all baseline)	X-Content-Type-Options: nosniff. X-Frame-Options: DENY. Referrer-Policy: strict-origin-when-cross-origin. Permissions-Policy: disables geolocation, camera, microphone, payment, USB, magnetometer, gyroscope. X-XSS-Protection: 0. Server: Sentinel. X-Permitted-Cross-Domain-Policies: none.	Every response inherits these defenses. Disables MIME sniffing, framing, cross-domain access, and unnecessary browser features. Replaces the Server header to hide the 4D version string.
^/api/dashboard/.*\$	Dashboard JSON API	Content-Security-Policy: default-src 'none', frame-ancestors 'none'. Cache-Control: no-store, no-cache, Pragma: no-cache. Access-Control-Allow-Origin: http://127.0.0.1:8044. Access-Control-Allow-Methods: GET, POST, OPTIONS. Access-Control-Allow-Headers: Content-Type, Authorization. Access-Control-Max-Age: 600. Vary: Origin. X-Robots-Tag: noindex, nofollow, noarchive.	Locks the dashboard JSON API to the same origin (127.0.0.1:8044). Prevents framing. Disables caching of sensitive admin responses. Tells search engines to ignore the path.

regexPattern	Applies to	Headers added	Why
<code>^/api/public/.*\$</code>	Public API	Content-Security-Policy: default-src 'none', frame-ancestors 'none'. Cache-Control: no-store, no-cache, must-revalidate. Access-Control-Allow-Origin: *. Access-Control-Allow-Methods: GET, POST, PUT, DELETE, OPTIONS. Access-Control-Allow-Headers: Content-Type. Access-Control-Max-Age: 600.	Allows cross-origin requests from any client (CORS wildcard) for the public endpoints. Disables caching. Strict CSP prevents the response body itself from loading anything if rendered in a browser.
<code>^/admin/.*\$</code>	Administrative endpoints	Content-Security-Policy: default-src 'none', frame-ancestors 'none'. Cache-Control: no-store, no-cache, must-revalidate, private, max-age=0. Pragma: no-cache. X-Robots-Tag: noindex, nofollow, noarchive.	Hides admin endpoints from search engines, blocks caching by browsers and proxies, prevents framing. CSP locks the response down to inert content.
<code>^/health\$</code>	Health-check endpoint	Cache-Control: no-store, max-age=0. Access-Control-Allow-Origin: *. X-Robots-Tag: noindex, nofollow.	Permissive CORS so monitoring tools can poll the health probe from any host. No caching so each poll reflects current server state.
<code>^/upload/.*\$</code>	File upload routes	Content-Security-Policy: default-src 'none'. X-Content-Type-Options: nosniff. Cache-Control: no-store. X-Robots-Tag: noindex, nofollow.	Strictest CSP for upload responses. Re-applies nosniff so an uploaded file can never be served back as executable script through MIME guessing.
<code>^/public/.*\$</code>		Cache-Control: public, max-age=300. Access-	Five-minute browser cache for

regexPattern	Applies to	Headers added	Why
	Public resources	Control-Allow-Origin: *. X-Robots-Tag: noindex, nofollow.	static public resources. Permissive CORS. Search engines are still asked to ignore the path.
^/(sentinel sentinel-config index dashboard)(\.?(html css js))?\$	Dashboard pages (login and admin UI)	Content-Security-Policy: default-src 'self'. style-src 'self' 'unsafe-inline' https://fonts.googleapis.com. script-src 'self' 'unsafe-inline'. font-src 'self' https://fonts.gstatic.com. img-src 'self' data:. connect-src 'self'. frame-ancestors 'none'. form-action 'self'. base-uri 'self'. Cache-Control: no-cache, must-revalidate. X-Robots-Tag: noindex, nofollow.	CSP allows the dashboard to load fonts from Google Fonts and inline styles and scripts (the dashboard ships with embedded CSS and JS). Blocks every other origin. Prevents framing, form action redirection, and base-URI hijacking.
^/\$	Bare root URL	Content-Security-Policy: same as the dashboard-pages rule above. Cache-Control: no-cache, must-revalidate. X-Robots-Tag: noindex, nofollow.	The bare root URL is treated like the dashboard pages. The same strict CSP applies. Search engines are asked to ignore it.

Response Headers Glossary

Every HTTP response header used anywhere in HTTPRules.json is defined below. One row per header.

Header	Definition	Where used in the demo application
X-Content-Type-Options	Tells the browser to trust the Content-Type declared by the server. Prevents MIME sniffing. The only valid value is nosniff.	Applied to every URL through the catch-all rule. Re-applied explicitly to /upload/*.

Header	Definition	Where used in the demo application
X-Frame-Options	Controls whether the page may be rendered inside an iframe. The value DENY blocks all framing. Defeats clickjacking.	Applied to every URL through the catch-all rule.
Referrer-Policy	Controls what the browser sends in the Referer header when the end user follows a link out of the page. The value strict-origin-when-cross-origin sends only the origin (no path) on cross-site navigation.	Applied to every URL through the catch-all rule.
Permissions-Policy	Disables browser features the page does not need. The demo application disables geolocation, camera, microphone, payment, USB, magnetometer, and gyroscope.	Applied to every URL through the catch-all rule.
X-XSS-Protection	Legacy header. Modern browsers honor CSP instead. The value 0 disables the legacy filter (which is known to introduce vulnerabilities).	Applied to every URL through the catch-all rule.
Server	The server identification string. Overwriting this header hides the underlying server software and version from clients and scanners.	Set to "Sentinel" through the catch-all rule, replacing the default 4D/21.0.0 string.
X-Permitted-Cross-Domain-Policies	Controls whether Adobe Flash and PDF readers may load cross-domain policy files. The value none disables this. Legacy but cheap to set.	Applied to every URL through the catch-all rule.
Content-Security-Policy (CSP)	The main response-side defense against cross-site scripting. Declares which origins are allowed to provide scripts, styles, fonts, images, and frames. A directive of 'none' means no origin is allowed for that resource kind.	Different rules send different CSP values. The dashboard pages get a permissive CSP that allows Google Fonts. API endpoints get default-src 'none'.
Cache-Control	Controls how browsers and proxies may cache the response. The value no-	Most defended rules send no-store.

Header	Definition	Where used in the demo application
	store forbids storage entirely. max-age=N limits cache lifetime to N seconds.	/public/* sends public, max-age=300 for static resources.
Pragma	Legacy header. Pragma: no-cache mirrors Cache-Control: no-cache for old HTTP/1.0 proxies.	Sent on /api/dashboard/* and /admin/* for defense in depth.
Access-Control-Allow-Origin	CORS. Declares which origins may read the response from JavaScript. The value * allows any origin. A specific URL pins to one origin.	Pinned to http://127.0.0.1:8044 for /api/dashboard/*. Set to * for public endpoints.
Access-Control-Allow-Credentials	CORS. Tells the browser whether cookies and HTTP authentication may be sent on the cross-origin request. Required to be true for cookie-based session auth.	Set to true only on /api/dashboard/* because the dashboard uses cookie-based session auth.
Access-Control-Allow-Methods	CORS. Lists the HTTP verbs the server accepts on the route. Returned in the preflight OPTIONS response.	GET, POST, OPTIONS on /api/dashboard/*. Wider list on /api/public/.
Access-Control-Allow-Headers	CORS. Lists the request headers the server accepts. Returned in the preflight OPTIONS response.	Content-Type and Authorization on /api/dashboard/.
Access-Control-Max-Age	CORS. Tells the browser how long it may cache the preflight result, in seconds.	Set to 600 (ten minutes) so the browser is not preflight-checking every JSON POST.
Vary	Tells caches that the response depends on a specific request header. Vary: Origin pairs with Access-Control-Allow-Origin to avoid serving the wrong CORS response from cache.	Set on /api/dashboard/.

Header	Definition	Where used in the demo application
X-Robots-Tag	Tells search-engine crawlers how to index the URL. The values noindex, nofollow, noarchive ask the crawler to not index, not follow links, and not archive a copy.	Applied to /api/dashboard/*, /admin/*, /upload/*, /public/*, /health, the dashboard pages, and the bare root.

Configuration and Persistence

File Layout

All runtime state lives directly under the project Data folder. Nesting state under Data/Settings has caused path-separator bugs on Windows and is forbidden.

File	Owner	Contents
Data/dosguard_config.json	ConfigManager	Defenses toggles, thresholds, bcrypt hash, WAF patterns.
Data/ip_lists.json	IPManager	Blocklist, allowlist, strike counters.
Data/Logs/dos_<date>.log	RequestLogger	Daily JSONL request log.
Data/sonar_kills.jsonl	Sonar_Sniper_Worker	Forensic record of one-hour blocks.
Data/sonar_archive.jsonl	Sonar_Drone_Worker	Pattern-correlation archive.
Data/dos_log.csv	Sentinel.handleExportLogs	Dashboard-exported CSV.

Atomic Write Pattern

ConfigManager and IPManager both use a write-then-rename pattern. The pattern prevents partial writes in case of a crash mid-write.

The sequence is. Write content to a temporary file. Delete the live file. Rename the temporary file to the live name.

Hot-Reload Semantics

ConfigManager reloads on demand through the dashboard /api/dashboard/config endpoint. No 4D server restart is required.

HTTPHandlers.json and HTTPRules.json are reloaded by the 4D web server itself when the file is rewritten. RulesManager and HandlersManager track the on-disk version for the dashboard.

Dashboard and Frontend

Once the Prerequisites and Requirements are met, open a web browser and navigate to either of the following URLs.

- <http://127.0.0.1:8044>
- <http://127.0.0.1:8044/sentinel.html>

When the login page appears, generate a passphrase and authenticate to access the dashboard.

For further instructions, refer to the README file in the project root.

Layout

The dashboard lives at <http://127.0.0.1:8044/dashboard.html>. The interface renders inside the 4D web server static-files folder and uses only vanilla HTML, CSS, and JavaScript.

- **Header.** Title, live clock, threat badge, density slider, theme toggle, logout.
- **Telemetry.** Eight metric cards. Total, blocked, rate limited, WAF, allowed, honeypot, panic rejected, requests per minute.
- **Defense layers.** Four toggle cards plus the master switch.
- **Intelligence.** CPU and traffic graphs plus IP blocklist controls.
- **Operations.** Workers, alerts, honeypot hits.
- **Controls.** Panic trigger, attack simulation, Security Equalizer entry, keyboard shortcuts.
- **Feed.** log lines with auto-scroll and CSV export.

Polling Cadence

The dashboard monitors four endpoints using independent timers. To optimize resource consumption when the browser tab is backgrounded, the application throttles the main polling frequency fivefold, pauses worker and honeypot requests entirely, and maintains the alert poll to queue critical notifications for immediate display upon focus restoration.

Endpoint	Cadence (visible)	Cadence (hidden)
/admin/status	1 second	5 seconds
/admin/workers	5 seconds	paused
/admin/honeypot/hits	10 seconds	paused
/api/dashboard/alerts	5 seconds	5 seconds

Auth-Required Handling

The JavaScript helper `api(url, method, body)` returns one of three values.

- Success. The parsed JSON body.
- HTTP 401 or 403. The `AUTH_REQUIRED` symbol.
- Network failure or HTTP 5xx response. The value `null`.

On `AUTH_REQUIRED`, polling stops and the login overlay is shown. Local state (logs, history) is cleared to avoid leaking the previous session.

Workers and Process Orchestration

Spawning

Workers are spawned by `ProcessOrchestrator.spawn`. Each worker registers a heartbeat slot inside the orchestrator.

Each worker runs an infinite loop. The loop body executes one duty cycle, then sleeps through `ProcessOrchestrator.sleep` which uses a `4D.Signal` under the hood.

Shutdown

On `Exit` calls `WEB Server.stop` first, then signals the shutdown semaphore, then issues `KILL WORKER` for each registered worker.

Workers wake from sleep on the signal, observe `ProcessOrchestrator.shouldShutdown`, break their loop, and exit cleanly.

Health Reporting

Each worker writes a timestamp to its heartbeat slot every loop iteration. The dashboard polls `/admin/workers` and compares slot ages against the expected cadence of each worker.

A worker whose heartbeat is older than three cadence intervals is shown as `STALLED` on the dashboard. `STALLED` also triggers a `WARNING`-level alert.

Conclusion

The demo application mitigates targeted threats across four categories: denial of service, injection, reconnaissance, and header attacks. Because the underlying threat model is extensible, developers can scale the application's defenses, for example, by adding a "Slowloris" guard to detect slow-read attacks via connection idle times, or a JA4 fingerprint check to feed TLS-level signals into the intrusion detection scoring. Architecturally, the application relies on two 4D v21.0 LTS features: HTTP Rules to enforce security headers on all web server responses, and HTTP Request Handlers to route incoming requests to custom classes, decoupling business logic from the routing layer.